Arcade Plugin Integration Guide

This document explains how to add new game and graphics libraries compatible with this Arcade architecture.

1. Runtime Architecture Summary

- The executable core loads dynamic libraries (.so) with dlopen and dlsym.
- A renderer library must implement Arc::IRenderer.
- A game library must implement Arc::IGame.
- Core shares one items container with the active game and renderer:
  - type: std::map<std::string, Arc::Item>
  - purpose: game writes scene state, renderer reads and draws it.

2. Required Exported Symbols

Every dynamic library must expose these C symbols:

- entryPoint
- getLibraryType
- getName

2.1 Game library exports

```cpp
extern "C" {
    Arc::IGame* entryPoint(std::map<std::string, Arc::Item>& items) {
        return new Arc::MyGame(items);
    }

    int getLibraryType() {
        return static_cast<int>(Arc::LibraryType::Game);
    }

    std::string getName() {
        return "MyGame";
    }
}
```

2.2 Renderer library exports

```cpp
extern "C" {
    Arc::IRenderer* entryPoint(std::map<std::string, Arc::Item>& items) {
        if (items.empty())
            return nullptr;
        return new Arc::MyRenderer();
    }

    int getLibraryType() {
        return static_cast<int>(Arc::LibraryType::Renderer);
    }

    std::string getName() {
        return "MyRenderer";
    }
}
```

3. Item Contract

The shared Arc::Item structure contains:

- position: world position (x, y)
- size: draw scale or text size control
- rotation
- textureSize: populated by renderer when useful
- isHovered
- drawOrder: z-order
- texturePath: sprite path (empty means no sprite)
- text: textual payload (for UI or ncurses)
- buttonAction: callback used by menu interactions

Practical convention used in this project:

- If texturePath is not empty, graphical renderers draw the sprite.
- If text is not empty and texturePath is empty, graphical renderers draw text.
- Hidden elements should use size = {0.0f, 0.0f}.

4. Implementing a New Game Library

4.1 Mandatory interface

Implement Arc::IGame:

- update(Event event)
- handleKey(Event event)
- getScore() const
- isGameOver() const

4.2 Minimal game skeleton

```cpp
#include "IGame.hpp"
#include "Item.hpp"
#include <map>

namespace Arc {
class MyGame : public IGame {
public:
    MyGame(std::map<std::string, Item>& items) : _items(items) {
        _items["Background"] = {{960, 540}, {1, 1}, 0, {0, 0}, false, -1,
"./assets/textures/menu/Menu_Background.png", "", nullptr};
        _items["Title"] = {{960, 540}, {40, 40}, 0, {0, 0}, false, 1, "", "MY
GAME", nullptr};
    }

    void update(Event event) override {
        (void)event;
    }

    void handleKey(Event event) override {
        if (event.type == Type::KeyPressed && event.code == Key::Escape)
            _backToMenu = true;
    }

    std::vector<int> getScore() const override { return {_score}; }
    bool isGameOver() const override { return _backToMenu; }

private:
    std::map<std::string, Item>& _items;
    int _score = 0;
    bool _backToMenu = false;
};
}
```

4.3 Game behavior expectations

- Do not own the render loop; Core handles loop and display.
- Read input in handleKey.
- Mutate _items in update and/or handleKey.
- Return true in isGameOver to go back to menu.

5. Implementing a New Renderer Library

5.1 Mandatory interface

Implement Arc::IRenderer:

- createWindow(int width, int height, const std::string& title)
- pollEvent(Event& event)
- display(std::map<std::string, Arc::Item>& items)

5.2 Event mapping rules

Map native framework inputs to Arc enums in Event.hpp:

- arrows -> Left, Right, Up, Down
- letters and numbers -> A..Z and Num0..Num9
- Enter, Backspace, Space, Tab
- mouse buttons -> MouseLeft, MouseRight
- quit signal -> Type::Quit

5.3 Display rules

- Sort items by drawOrder ascending before drawing.
- Draw Background first (usually drawOrder = -1).
- Respect hidden items (size <= 0 on any axis means not drawn).
- Keep coordinate conventions consistent with existing renderers.

6. Build and Placement Rules

- Build plugins as shared objects:
  - g++ -shared -fPIC ... -o lib/arcade_mygame.so
  - g++ -shared -fPIC ... -o lib/arcade_myrenderer.so
- Keep exported symbols unmangled with extern "C".
- Place resulting files in lib.

7. Core Discovery and Launch Flow

- Main verifies the initial argument is a renderer library with getLibraryType.
- Core stores available renderer and game paths.
- Menu lets the user choose game and renderer.
- Core::launchGame may switch renderer, then loads the selected game.

8. Integration Checklist

Before turn-in, verify:

- Library loads with no dlopen/dlsym errors.
- getLibraryType matches the plugin type.
- getName returns a clean visible name.
- Input works through pollEvent mapping.
- Scene appears through shared items map.
- Hidden items stay hidden in graphical backends.
- Escape and quit flows are handled safely.

9. Troubleshooting

- Crash at load time:

  - check symbol names and extern "C".
  - ensure function signatures match exactly.
- Black screen:
  - verify items are created and have valid size.
  - ensure renderer display iterates through items.
- Wrong input behavior:
  - check native key to Arc::Key mapping.
- Artifacts when switching renderer:
  - enforce visibility check (size > 0 on both axes) before drawing.